

Question:

Write a JAVA program to demonstrate the interfaces of the Java Collections Framework: **List, Set, SortedSet, NavigableSet, Queue, Deque, Map, SortedMap, NavigableMap, Iterator and ListIterator**, using their commonly used methods.

Execution Steps:

Step 1: Program execution starts from main().

Step 2: **List**: an ArrayList is created through a List reference and "Java", "Python", "C", "Java" are added. size() gives 4, contains("Java") gives true and get(1) gives Python.

Step 3: add(1, "C++") inserts at index 1 and set(2, "JavaScript") replaces the element at index 2. indexOf("Java") gives 0 and lastIndexOf("Java") gives 4. remove("C") removes the first match and subList(0, 2) gives [Java, C++].

Step 4: **Set**: a HashSet stores 10, 20, 30 and the duplicate 20 is ignored, so size() is 3. After remove(10) the set is [20, 30].

Step 5: **SortedSet**: a TreeSet keeps its elements sorted: [10, 20, 30, 40]. first() = 10, last() = 40, headSet(30) = [10, 20], tailSet(20) = [20, 30, 40] and subSet(20, 40) = [20, 30].

Step 6: **NavigableSet**: for the value 30, lower() = 20, floor() = 30, ceiling() = 30 and higher() = 40. descendingSet() gives [50, 40, 30, 20, 10].

Step 7: **Queue**: a LinkedList is used as a FIFO queue. After add("A"), offer("B") and offer("C"), peek() and element() return A without removing it. poll() removes A and the queue becomes [B, C].

Step 8: **Deque**: an ArrayDeque gets addFirst(20), addLast(30), offerFirst(10) and offerLast(40), giving [10, 20, 30, 40]. peekFirst() = 10 and peekLast() = 40. removeFirst() and removeLast() leave [20, 30].

Step 9: **Map**: a HashMap stores the keys 101, 102 and 103 with their names. get(101) gives Deekshita, containsKey(102) and containsValue("Anu") are true, and keySet(), values() and entrySet() are displayed. remove(103) leaves two entries.

Step 10: **SortedMap**: a TreeMap keeps the keys sorted: {1=One, 2=Two, 3=Three, 4=Four}. firstKey() = 1, lastKey() = 4, headMap(3) = {1, 2}, tailMap(2) = {2, 3, 4} and subMap(2, 4) = {2, 3}.

Step 11: **NavigableMap**: for the key 30, lowerKey() = 20, floorKey() = 30, ceilingKey() = 30 and higherKey() = 40. firstEntry() = 10=Ten, lastEntry() = 40=Forty and descendingMap() reverses the order.

Step 12: **Iterator**: iterator() is obtained from the list [10, 20, 30, 40] and the while loop with hasNext() and next() prints 10 20 30 40.

Step 13: **ListIterator**: the list [A, B, C] is traversed forward with hasNext() and next(): A B C, and then backward with hasPrevious() and previous(): C B A. The list itself remains [A, B, C].

Step 14: main() ends and the program terminates.

Algorithm:

Step 1: Start.

Step 2: Import the java.util package.

Step 3: List: create an ArrayList, add elements, and use size(), contains(), get(), add(index), set(), indexOf(), lastIndexOf(), remove() and subList().

Step 4: Set: create a HashSet, add elements (duplicates are ignored) and use size(), contains() and remove().

Step 5: SortedSet: create a TreeSet and use first(), last(), headSet(), tailSet() and subSet().

Step 6: NavigableSet: use lower(), floor(), ceiling(), higher() and descendingSet().

Step 7: Queue: create a LinkedList as a Queue and use add(), offer(), peek(), element() and poll().

Step 8: Deque: create an ArrayDeque and use addFirst(), addLast(), offerFirst(), offerLast(), peekFirst(), peekLast(), removeFirst() and removeLast().

Step 9: Map: create a HashMap and use put(), get(), containsKey(), containsValue(), keySet(), values(), entrySet(), size() and remove().

Step 10: SortedMap: create a TreeMap and use firstKey(), lastKey(), headMap(), tailMap() and subMap().

Step 11: NavigableMap: use lowerKey(), floorKey(), ceilingKey(), higherKey(), firstEntry(), lastEntry() and descendingMap().

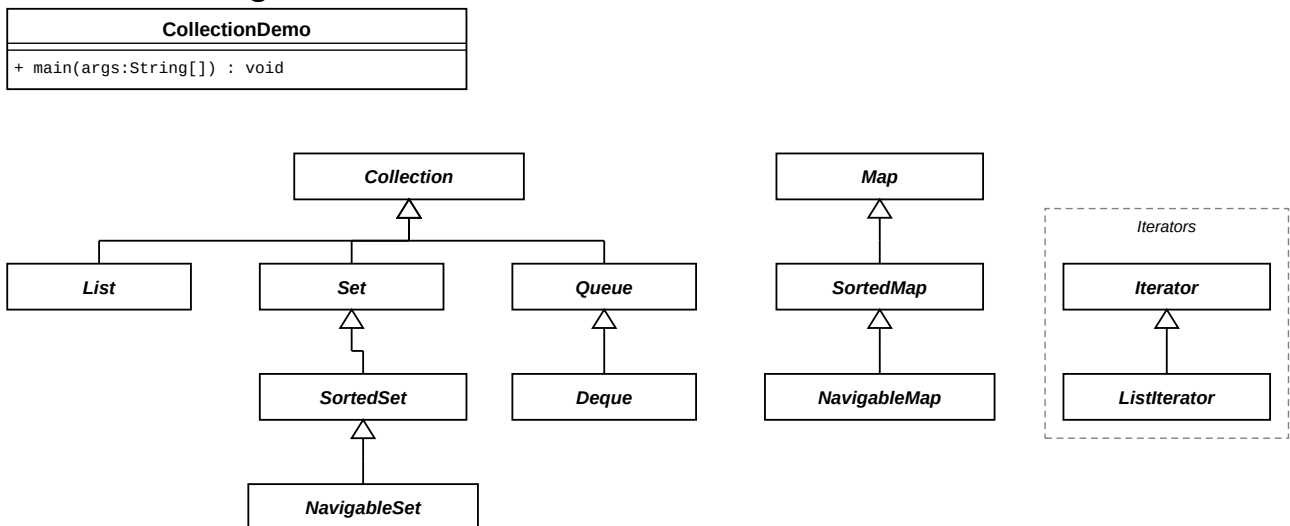
Step 12: Iterator: traverse a list using hasNext() and next().

Step 13: ListIterator: traverse a list forward and then backward using hasNext(), next(), hasPrevious() and previous().

Step 14: Display the result after each operation.

Step 15: Stop.

UML Class Diagram:



The diagram shows the Java Collections Framework interfaces used in the program (java.util) together with the demo class. A solid line with a hollow triangle means one interface extends another. Iterator and ListIterator form a separate hierarchy. Objects used in the program: ArrayList (List), HashSet (Set), TreeSet (SortedSet and NavigableSet), LinkedList (Queue), ArrayDeque (Deque), HashMap (Map), TreeMap (SortedMap and NavigableMap).

Code:

```
import java.util.*;

public class CollectionDemo {
    public static void main(String[] args) {

        // ----- COLLECTION / LIST -----
        System.out.println("==== LIST INTERFACE =====");

        List<String> list = new ArrayList<>();

        list.add("Java");
        list.add("Python");
        list.add("C");
        list.add("Java");

        System.out.println("List: " + list);
        System.out.println("Size: " + list.size());
        System.out.println("Contains Java: " + list.contains("Java"));
        System.out.println("Element at index 1: " + list.get(1));

        list.add(1, "C++");
        System.out.println("After add(index): " + list);

        list.set(2, "JavaScript");
        System.out.println("After set(): " + list);

        System.out.println("First index of Java: " + list.indexOf("Java"));
        System.out.println("Last index of Java: " + list.lastIndexOf("Java"));

        list.remove("C");
        System.out.println("After remove(): " + list);

        System.out.println("SubList: " + list.subList(0, 2));

        // ----- SET -----
        System.out.println("\n==== SET INTERFACE =====");

        Set<Integer> set = new HashSet<>();

        set.add(10);
        set.add(20);
        set.add(30);
        set.add(20); // Duplicate ignored

        System.out.println("Set: " + set);
        System.out.println("Size: " + set.size());
        System.out.println("Contains 20: " + set.contains(20));

        set.remove(10);
        System.out.println("After remove(10): " + set);

        // ----- SORTED SET -----
        System.out.println("\n==== SORTED SET INTERFACE =====");

        SortedSet<Integer> sortedSet = new TreeSet<>();

        sortedSet.add(40);
        sortedSet.add(10);
        sortedSet.add(30);
        sortedSet.add(20);

        System.out.println("Sorted Set: " + sortedSet);
        System.out.println("First: " + sortedSet.first());
        System.out.println("Last: " + sortedSet.last());
        System.out.println("HeadSet(30): " + sortedSet.headSet(30));
        System.out.println("TailSet(20): " + sortedSet.tailSet(20));
        System.out.println("SubSet(20, 40): " + sortedSet.subSet(20, 40));
        // ----- NAVIGABLE SET -----
```

```

System.out.println("\n===== NAVIGABLE SET INTERFACE =====");

NavigableSet<Integer> navSet = new TreeSet<>();

navSet.add(10);
navSet.add(20);
navSet.add(30);
navSet.add(40);
navSet.add(50);

System.out.println("Navigable Set: " + navSet);
System.out.println("Lower(30): " + navSet.lower(30));
System.out.println("Floor(30): " + navSet.floor(30));
System.out.println("Ceiling(30): " + navSet.ceiling(30));
System.out.println("Higher(30): " + navSet.higher(30));
System.out.println("Descending Set: " + navSet.descendingSet());

// ----- QUEUE -----
System.out.println("\n===== QUEUE INTERFACE =====");

Queue<String> queue = new LinkedList<>();

queue.add("A");
queue.offer("B");
queue.offer("C");

System.out.println("Queue: " + queue);
System.out.println("Peek: " + queue.peek());
System.out.println("Element: " + queue.element());

queue.poll();
System.out.println("After poll(): " + queue);

// ----- DEQUE -----
System.out.println("\n===== DEQUE INTERFACE =====");

Deque<Integer> deque = new ArrayDeque<>();

deque.addFirst(20);
deque.addLast(30);
deque.offerFirst(10);
deque.offerLast(40);

System.out.println("Deque: " + deque);
System.out.println("Peek First: " + deque.peekFirst());
System.out.println("Peek Last: " + deque.peekLast());

deque.removeFirst();
deque.removeLast();

System.out.println("After removing first and last: " + deque);

// ----- MAP -----
System.out.println("\n===== MAP INTERFACE =====");

Map<Integer, String> map = new HashMap<>();

map.put(101, "Deekshita");
map.put(102, "Anu");
map.put(103, "Priya");

System.out.println("Map: " + map);
System.out.println("Value for key 101: " + map.get(101));
System.out.println("Contains key 102: " + map.containsKey(102));
System.out.println("Contains value Anu: " + map.containsValue("Anu"));
System.out.println("Keys: " + map.keySet());
System.out.println("Values: " + map.values());
System.out.println("Entries: " + map.entrySet());
System.out.println("Size: " + map.size());

```

```

map.remove(103);
System.out.println("After remove(103): " + map);

// ----- SORTED MAP -----
System.out.println("\n==== SORTED MAP INTERFACE =====");

SortedMap<Integer, String> sortedMap = new TreeMap<>();

sortedMap.put(3, "Three");
sortedMap.put(1, "One");
sortedMap.put(2, "Two");
sortedMap.put(4, "Four");

System.out.println("Sorted Map: " + sortedMap);
System.out.println("First Key: " + sortedMap.firstKey());
System.out.println("Last Key: " + sortedMap.lastKey());
System.out.println("Head Map(3): " + sortedMap.headMap(3));
System.out.println("Tail Map(2): " + sortedMap.tailMap(2));
System.out.println("Sub Map(2,4): " + sortedMap.subMap(2, 4));

// ----- NAVIGABLE MAP -----
System.out.println("\n==== NAVIGABLE MAP INTERFACE =====");

NavigableMap<Integer, String> navMap = new TreeMap<>();

navMap.put(10, "Ten");
navMap.put(20, "Twenty");
navMap.put(30, "Thirty");
navMap.put(40, "Forty");

System.out.println("Navigable Map: " + navMap);
System.out.println("Lower Key(30): " + navMap.lowerKey(30));
System.out.println("Floor Key(30): " + navMap.floorKey(30));
System.out.println("Ceiling Key(30): " + navMap.ceilingKey(30));
System.out.println("Higher Key(30): " + navMap.higherKey(30));
System.out.println("First Entry: " + navMap.firstEntry());
System.out.println("Last Entry: " + navMap.lastEntry());
System.out.println("Descending Map: " + navMap.descendingMap());

// ----- ITERATOR -----
System.out.println("\n==== ITERATOR INTERFACE =====");

List<Integer> numbers = new ArrayList<>();

numbers.add(10);
numbers.add(20);
numbers.add(30);
numbers.add(40);

Iterator<Integer> iterator = numbers.iterator();

System.out.print("Elements using Iterator: ");

while (iterator.hasNext()) {
    System.out.print(iterator.next() + " ");
}

System.out.println();

// ----- LIST ITERATOR -----
System.out.println("\n==== LIST ITERATOR INTERFACE =====");

List<String> names = new ArrayList<>();

names.add("A");
names.add("B");
names.add("C");

ListIterator<String> listIterator = names.listIterator();

```

```
        System.out.print("Forward direction: ");

        while (listIterator.hasNext()) {
            System.out.print(listIterator.next() + " ");
        }

        System.out.print("\nBackward direction: ");

        while (listIterator.hasPrevious()) {
            System.out.print(listIterator.previous() + " ");
        }

        System.out.println();
        System.out.println("Final List: " + names);
    }
}
```

Output:

```
===== LIST INTERFACE =====
List: [Java, Python, C, Java]
Size: 4
Contains Java: true
Element at index 1: Python
After add(index): [Java, C++, Python, C, Java]
After set(): [Java, C++, JavaScript, C, Java]
First index of Java: 0
Last index of Java: 4
After remove(): [Java, C++, JavaScript, Java]
SubList: [Java, C++]

===== SET INTERFACE =====
Set: [20, 10, 30]
Size: 3
Contains 20: true
After remove(10): [20, 30]

===== SORTED SET INTERFACE =====
Sorted Set: [10, 20, 30, 40]
First: 10
Last: 40
HeadSet(30): [10, 20]
TailSet(20): [20, 30, 40]
SubSet(20,40): [20, 30]

===== NAVIGABLE SET INTERFACE =====
Navigable Set: [10, 20, 30, 40, 50]
Lower(30): 20
Floor(30): 30
Ceiling(30): 30
Higher(30): 40
Descending Set: [50, 40, 30, 20, 10]

===== QUEUE INTERFACE =====
Queue: [A, B, C]
Peek: A
Element: A
After poll(): [B, C]

===== DEQUE INTERFACE =====
Deque: [10, 20, 30, 40]
Peek First: 10
Peek Last: 40
After removing first and last: [20, 30]

===== MAP INTERFACE =====
Map: {101=Deekshita, 102=Anu, 103=Priya}
Value for key 101: Deekshita
Contains key 102: true
Contains value Anu: true
Keys: [101, 102, 103]
Values: [Deekshita, Anu, Priya]
Entries: [101=Deekshita, 102=Anu, 103=Priya]
Size: 3
After remove(103): {101=Deekshita, 102=Anu}

===== SORTED MAP INTERFACE =====
Sorted Map: {1=One, 2=Two, 3=Three, 4=Four}
First Key: 1
Last Key: 4
Head Map(3): {1=One, 2=Two}
Tail Map(2): {2=Two, 3=Three, 4=Four}
```

```
Sub Map(2,4): {2=Two, 3=Three}
```

```
===== NAVIGABLE MAP INTERFACE =====
```

```
Navigable Map: {10=Ten, 20=Twenty, 30=Thirty, 40=Forty}
```

```
Lower Key(30): 20
```

```
Floor Key(30): 30
```

```
Ceiling Key(30): 30
```

```
Higher Key(30): 40
```

```
First Entry: 10=Ten
```

```
Last Entry: 40=Forty
```

```
Descending Map: {40=Forty, 30=Thirty, 20=Twenty, 10=Ten}
```

```
===== ITERATOR INTERFACE =====
```

```
Elements using Iterator: 10 20 30 40
```

```
===== LIST ITERATOR INTERFACE =====
```

```
Forward direction: A B C
```

```
Backward direction: C B A
```

```
Final List: [A, B, C]
```